

DNA Foundation Models in DeepChem Using Transformers

Abstract.

This project aims to extend DeepChem by introducing transformer-based foundation models for DNA sequence data. While DeepChem provides strong support for molecular machine learning, it lacks integrated support for large-scale biological sequence modeling.

The proposed solution leverages transformer architectures from Hugging Face Transformers to design a DNA sequence model using k-mer tokenization. The model will be integrated into DeepChem via the TorchModel interface, enabling seamless training, evaluation, and deployment. The project includes pretraining using masked language modeling (MLM), followed by fine-tuning on downstream biological tasks such as sequence classification. The final system will be benchmarked on standard datasets.

Introduction and Motivation:

Recent advances in transformer-based models have significantly improved sequence modeling tasks in natural language processing. Similar approaches have shown promise in biological domains, where DNA sequences can be treated analogously to text. Despite this progress, DeepChem does not yet provide native support for transformer-based foundation models tailored to biological sequences. Integrating such models would enable researchers to perform scalable pretraining and transfer learning on genomic data. This project aims to bridge that gap by developing a reusable and extensible framework for DNA-based foundation models within DeepChem.

Problem Statement:

DeepChem currently lacks:

- Native support for DNA sequence tokenization
- Transformer-based foundation models for biological data
- Pretraining pipelines for sequence modeling

The goal is to:

- Develop a DNA tokenizer
- Build a transformer-based model
- Integrate with DeepChem's training ecosystem
- Enable pretraining and fine-tuning workflows

Project Goals and Deliverables:

Primary Goals

- Implement a DNA tokenizer using k-mer representation
- Build a transformer-based DNA model
- Integrate model into DeepChem via TorchModel

Secondary Goals

- Implement masked language modeling for pretraining
- Fine-tune model for classification tasks
- Benchmark performance on biological datasets

Final Deliverables

- Tokenizer module integrated into DeepChem
- Transformer model implementation
- Training and pretraining pipelines
- Benchmark results and evaluation metrics
- Documentation and usage examples

Technical Approach:

1. Data Representation

DNA sequences (e.g., ATCG) will be converted into k-mer tokens.

Example: Sequence: ATCGGCTA

k=3 → ATC, TCG, CGG, GGC, GCT, CTA

2. Tokenizer Design

A custom tokenizer will:

- Generate k-mer tokens
- Build a vocabulary
- Map tokens to integer IDs
- Support padding and batching

3. Model Architecture

- Encoder: BERT-style model
- Configurable parameters:
 - Hidden size
 - Attention heads
 - Number of layers
- The model will be implemented using Hugging Face Transformers.

4. DeepChem Integration

The model will be wrapped using: `dc.models.TorchModel`. This ensures compatibility with:

- Training (`fit`)
- Prediction (`predict`)
- Dataset handling

5. Pretraining Strategy

Masked Language Modeling (MLM):

- Randomly mask tokens
- Train model to predict masked tokens

This enables learning of sequence patterns.

6. Fine-Tuning

The pretrained model will be fine-tuned for: DNA classification tasks

7. Evaluation

Metrics are Accuracy, F1-score and ROC-AUC

Datasets: Public DNA classification datasets

Implementation Plan:

Phase 1: Set up environment and study DeepChem

Phase 2: Implement DNA tokenizer and prepare dataset pipeline

Phase 3: Build transformer model and integrate with DeepChem

Phase 4: Implement training pipeline and perform fine-tuning

Phase 5: Add pretraining (MLM) and evaluate the performance

Phase 6: Documentation and final testing

Timeline (12 Weeks):

Community Bonding: Understand DeepChem and Hugging Face Transformers and Finalize dataset and design

Weeks 1–2: Implement DNA tokenizer and dataset preparation

Weeks 3–5: Build transformer model and DeepChem integration

Weeks 6–8: Training and fine-tuning and Initial evaluation

Weeks 9–10: Implement MLM pretraining and performance improvements

Weeks 11–12: Benchmarking and Documentation and final submission

Evaluation Plan:

The project will be evaluated based on:

- Correct integration with DeepChem
- Model performance on benchmarks
- Code quality and modularity
- Documentation clarity

Why This Project Matters:

This project extends DeepChem into biological sequence modelling, enables transformer-based learning for genomics and bridges NLP and bioinformatics

Future Work:

- Support long-sequence models
- Extend to single-cell data
- Add explainability methods

Conclusion:

This project introduces transformer-based DNA foundation models into DeepChem, enabling scalable biological sequence modeling. By combining modern transformer architectures with DeepChem's ecosystem, the project enhances its applicability in bioinformatics and computational biology.

About Me:

I am **Inzeera Z**, currently pursuing III Year Engineering student who aspires to be an AI Engineer having a hands-on experience in building and working with transformer-based models using Hugging Face Transformers. I have implemented projects involving model loading, tokenization, training, and inference workflows, which has given me a solid understanding of transformer architectures and sequence modeling.

I am also familiar with deep learning frameworks such as PyTorch, tensorflow and have experience handling data preprocessing, model training, and evaluation pipelines. My work has involved applying machine learning and deep learning techniques to real-world problems, which has strengthened my ability to design and implement end-to-end solutions.

In addition, I have experience working with structured and unstructured data, and I am comfortable with tools such as NumPy, Pandas, and scikit-learn for data processing and analysis.

Through this project, I aim to deepen my understanding of integrating transformer models into domain-specific frameworks like DeepChem and contribute meaningful functionality to the open-source community.